USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

I²C, and Why a Preempted Master Is Legal

`UM-NATOS-025`

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-025	1.0 · 2026-08-16	**Bus verified electrically; protocol unexercised**	Internal

1. Abstract

The ADC turned one pin into one sensor. This turns two pins into **most** sensors — temperature, humidity, pressure, accelerometers, magnetometers, real-time clocks, small displays — and they share the bus, so the pin cost does not grow with the sensor count. On a board with three spare pins that matters more than it would elsewhere.

The driver is ordinary. Two things in it are worth a report: the reason it is bit-banged is a property of the *protocol* rather than of this code being careful, and the failure mode it guards against is one where **everything appears to work perfectly**.

2. Bit-banged, and why that is safe here

This part has two hardware I²C controllers. Neither is used.

The weak reason is consistency: the display and touch drivers are both bit-banged, so the technique is proven in this kernel and its failure modes are understood.

The real reason is that **clock stretching is in the I²C specification**. A slave may hold SCL low to buy itself time, and every master is required to tolerate it. A bus that is legal to stall is therefore a bus on which a *preempted master* is legal too — a task switch in the middle of a transaction stretches the clock rather than corrupting the transfer.

> That is a guarantee from the protocol, not a consequence of this code being careful. It is also exactly what bit-banged I²S would not have, which is why UM-NATOS-023 §8's sequencing puts audio behind working interrupts and DMA while I²C needs neither.

`scl_release_and_wait()` is where this lives: the master never assumes the clock rose because it let go of it. It waits for SCL to actually read high, with a bounded timeout so a genuinely stuck bus is a reported error rather than a hang inside a shell command.

3. Open drain, emulated

I²C devices only ever pull **down**. A line is:

- **driven low** by enabling the output driver, which permanently holds zero
- **released** by disabling the driver, letting a pull-up raise the line

The output register is written once at init and never again; only the enable is toggled. Nothing is ever driven high, because two masters — or a slave mid-ACK — would then be shorted against each other rather than merely contending.

`FUN_IE` is set on both pads. An open-drain pin whose input buffer is off reads a constant zero, which on this bus looks exactly like every device in the world ACKing at once. That is the same class of defect as the touch controller reading silence as maximum pressure (UM-NATOS-017 §3.2).

4. The pull-up compromise

The internal pull-ups are used: roughly **45 kΩ**, against the 2.2–10 kΩ a real bus wants.

This is a stated compromise, not a design. Rise times are slow, so the clock is slow to match — a master that clocks faster than the line can rise reads its own transition times as data. Nothing here needs speed: a sensor read is a few dozen bytes and the caller is a task that sleeps anyway. Most breakout modules carry their own pull-ups and will be faster.

5. The failure that looks like success

A bus with no pull-up reports every address as present.

SDA floats, reads low, and a floating low is indistinguishable from an ACK. A scan then returns 112 devices, which is not a subtle wrong answer — but the individual result at any one address is entirely plausible, and code that probes a single expected sensor would find it and proceed.

`i2c_selftest()` checks this before any scan is believed, per line and in both directions, so that **"the pull-up works" and "the driver can pull down" are separate results**:

```
SDA (gpio22)  released=HIGH  driven=LOW    ok
SCL (gpio27)  released=HIGH  driven=LOW    ok
bus looks sane; a scan can be believed
scanning 0x08..0x77
0 device(s)
(nothing attached is the expected result on a bare board)
```

Both halves matter. A line that reads high when released and high when driven means the driver is not working; a line that reads low both times means there is no pull-up. Only one combination is a working line, and it is the one that gets the word `ok`.

The scan also reports a count over 100 as a stuck SDA rather than as devices, because that is what it is.

6. Pins

Signal	GPIO	Why
SDA	22	brought out to a header, unclaimed
SCL	27	brought out to a header, unclaimed

Everything else on this board is spoken for: display (2, 12–15, 21), touch (25, 32, 33, 36, 39), SD (5, 18, 19, 23), light sensor (34), speaker (26), RGB LED (4, 16, 17).

7. What is verified

- **The electrical layer, completely.** Both lines pull high when released and low when driven, measured, with no device attached.
- **A scan runs to completion** and correctly reports nothing on a bare bus.

8. What this does not establish

- **No byte has ever been transferred.** START, STOP, the ACK bit, repeated START and the read path are all correct by inspection and have never moved data. The first real device will be the first test of §9's logic, and the most likely defects are the ones inspection is worst at: a half-bit of timing, or an ACK sampled on the wrong clock edge.
- **Clock stretching has never happened.** The handler that justifies the whole bit-banging argument is unexercised, because nothing has ever stretched.
- **Speed is unmeasured.** The delay was chosen to be conservative against a 45 kΩ pull-up, not measured against a scope.
- **No multi-master arbitration.** The driver assumes it is the only master. It does not check that SDA read back what it drove, so it cannot detect losing arbitration.
- **No bus recovery.** A slave left mid-transfer can hold SDA low forever; the standard remedy is to clock SCL nine times to flush it, and that is not implemented.
- **No applications can reach it**, like every other peripheral here (UM-NATOS-022 §2).

9. References

- UM-NATOS-017 §3.2 — silence read as a valid measurement, the same class of defect as §3's `FUN_IE`
- UM-NATOS-023 §8 — why audio needs interrupts and DMA while this needs neither
- UM-NATOS-024 — the ADC, and the discipline of fetching constants rather than recalling them, which this driver's pad definitions follow
- `kernel/i2c.c` — the driver, the self-test and the scan